招新面试问题清单

> 设计原则：**不考他们现在会不会机器人**，只考三件事——
> **基础知识扎不扎实、遇到问题会不会找人、有没有自己动手做过东西。**
> 所有技术细节都可以进来再学，但「怎么 debug」这个习惯很难速成。
>
> 单人面试建议 15~20 分钟，从 A 类挑 1 题、B 类挑 2 题、C 类挑 2~3 题、
> E 类挑 1 题。**C 类是重点，时间不够就砍 B 类。**

A. 破冰与动机（2~3 分钟，挑 1 题）

A1. 你最近自己动手做过什么「能跑起来」的东西?
随便什么都行——脚本、小工具、游戏、网页、单片机小玩意。

> 想听到：能说出**具体做了什么、遇到什么坑、怎么解决的。**
> 减分：只说「学过某门课 / 看过某本书 / 跟着教程敲了一遍」，说不出自己改过什么。
> 追问：「中间卡得最久的一次是什么？怎么过去的？」

A2. 你为什么想进这个队?

> 想听到：对机器人/AI/嵌入式/竞赛有具体的兴趣来源（比如拆过什么、做过什么项目）。
> 减分：只回答「想拿奖」「想加学分」「听说很厉害」。
> 说明：这个答案本身不算错，但**只**有这个答案就说明动机不稳，后面容易掉队。

A3. 你更想做哪一块：写代码、调硬件、做算法、还是做整体方案?

> 想听到：有偏好但也愿意都碰。
> 注意：不要因为他说「想做大模型」就淘汰，也不要因为他说「我只会一点 C」就淘汰——
> 我们队里最需要的是**愿意把东西调通**的人，不是知识最多的人。

B. 基础知识（5~8 分钟，挑 2 题；全部只需口头回答）

> 这一节的目的是**筛掉完全零基础又不愿学的人**，不是考八股。
> 答不上来但能说清「我知道大概是什么方向，具体命令我查一下」的，给分。

B1 ~ B4：Linux 与命令行

B1. 怎么在一个文件夹里找出所有内容包含某个关键词的文件?

> 参考：`grep -r 关键词 目录`。加分项：还知道 `-i` 忽略大小写、`-n` 显示行号。

B2. 程序打印「Permission denied」，你会先看什么?

> 想听到：看文件权限、有没有执行权限、是不是要 `sudo`、是不是挂载盘不允许执行。

B3. 你的程序要连续跑一整晚，但一关终端就没了，怎么办?

> 想听到：`nohup ... &`、`screen` / `tmux`、或者写成服务。
> 不要求答全，知道「不能直接关终端」就算过。

B4. 怎么知道一个程序现在占了多少 CPU 和内存?

> 参考：`top` / `htop` / `ps aux`。

B5 ~ B8：Python 基础

B5. 列表 `a = [1, 2, 3]`，我写 `b = a` 然后改 `b[0] = 9`，`a` 会变吗？为什么？

> **答案要点**：会变。Python 的变量不是「盒子」，而是贴在对象上的「名字」。
> `b = a` 没有复制列表，只是给**同一个对象**又起了一个名字，所以改 `b[0]` 就是改那个对象。
>
> **要能区分两件事**：

```
a = [1, 2, 3]
b = a
b[0] = 9        # 改【对象的内容】→ a 跟着变，输出 [9, 2, 3]

a = [1, 2, 3]
b = a
b = [9, 2, 3]   # 给 b 换了个新对象 → a 不变，输出 [1, 2, 3]
```

> 想复制得写 `b = a.copy()` / `b = a[:]` / `b = list(a)`。
> 而 `int`、`str`、`tuple` 是**不可变**的，所以「就地改」根本不存在，看起来就没这个问题。
>
> 这题很重要——**它暴露的是「有没有真正理解变量」**，我们路径生成脚本里 `[row.copy() for row in base_rows[158:]]` 这种写法就是在防这个坑。
>
> **追问（用来分辨真懂还是背的）**：
>
> 1. 把那句改成 `b = [9, 2, 3]`，`a` 还变吗？（考「改对象」vs「改名字」）
> 2. 列表里装的还是列表呢？（答得出「浅拷贝不够、要用 `copy.deepcopy`」的加分）
> 3. 你在什么场景下真的被这个坑过？（能讲出具体故事的最可信）
>
> 答不上来但提示一句就通的，也是好苗子——这题本来就是用来观察学习速度的。

B6. 写一行代码，从 `[3, 1, 4, 1, 5]` 里把大于 2 的数挑出来。

> 参考：`[x for x in nums if x > 2]`，或 `filter`。

B7. 一个函数要返回两个值，怎么写？

> 参考：`return a, b`（本质是返回元组），调用处 `x, y = f()`。

B8. 你写了个循环处理 10 万条数据，跑得特别慢，你会先想什么？

> 想听到：先确认**慢在哪里**（加计时/打印），再想优化；不要一上来就换语言。
> 加分：知道「在循环里反复查数据库 / 反复读文件」是大忌。

B9 ~ B11：数学与逻辑

B9. 给一组测量值，怎么判断「这组数据跳得厉不厉害」？

> 想听到：平均值、最大值最小值、**标准差/方差**。答出「先画个图看看」也很好。

B10. 向量点积的几何意义是什么？

> 想听到：两个向量的相似程度 / 夹角。加分：能说出点积为 0 表示垂直。

B11. 为什么不能直接写 `if (0.1 + 0.2 == 0.3)`？

> 想听到：浮点数有精度误差，要比较差值是否小于一个很小的数。
> 提示：如果他答不上来但能理解，讲一遍即可，这题用来观察学习速度。

B12：网络常识

B12. 两台电脑上的程序要互相发消息，为什么需要知道对方的 IP 和端口?

> 想听到：IP 找到哪台机器，端口找到机器上的哪个程序。
> （我们车端和 PC 之间的大模型通信就是这个模式。）

C. Debug 思维（8~10 分钟，核心部分，挑 2~3 题）

> 这一节我不问「你会不会 X」，我问**「你会怎么查」**。答题过程比答案重要。
>
> 每道题后面都标了 **【出处】**，全部是我们这个项目真实踩过的坑。
> 这样你能判断「什么样的回答说明他真的下过功夫」，而不是背过概念。
>
> **核心题 C1~C5**：门槛低、区分度高，建议每个人都问 1~2 道。
> **进阶题 C6~C10**：留给简历答得顺的人。**答不出来不减分**，看的是他遇到陌生问题时的反应。

C1.「昨天还能跑，今天不动了」【核心 · 最推荐】

> 代码没改，硬件没动，但今天一跑就是不动。你会按什么顺序排查?

> **想听到**（不要求说全，出现 3 条以上就很好）：
>
> 1. **先确认「没改」是不是真的**——用版本对比、看文件修改时间，而不是靠记忆;
> 2. 从**最外层**查起：供电了吗？线插好了吗？程序真的启动了吗?
> 3. 看**输入**对不对（传感器有没有数据），再看处理，最后看输出;
> 4. 加打印/日志，把「黑盒」变成「看得见每一步」;
> 5. 一次只改一个变量。
>
> **减分**：直接说「那我重装一下」「重启试试」「肯定是硬件坏了」——不是不能试，
> 但要能说出**为什么这样试能带来信息**。
>
> 【出处】这类问题我们几乎每周都会遇到一次，是最高频的一种。

C2.「十次里错一次」【核心】

> 有个 bug 十次里只出现一次，重启就又好了。你会怎么办?

> **想听到**:
>
> 1. 想清楚「什么条件会触发它」——是不是和温度、速度、时间、数据量有关;
> 2. **想办法提高复现概率**（反复跑、加大数据量、缩短周期），而不是等它自己出现;
> 3. 复现不了就先**加日志**，等下次出现时留下证据。
>
> **减分**：「那就重启一下」。
>
> 【出处】我们的程序在连续跑得比较久的时候会突然卡半拍，重启就好。
> 当时一直抓不到，后来加了「每 5 秒打印一次队列长度」才看清是数据积压。
> **偶发问题基本不可能靠盯着看查出来，只能靠留下证据。**

C3.「报错看不懂」【核心】

> 程序报了一大段错误，你只认识最后一个单词 "failed"。你下一步做什么？

> **想听到**：**从第一行错误开始读**（后面的往往是被前面连累的）；
> 把报错原文粘到搜索引擎；看是哪个文件哪一行；
> 分辨「这是程序自己打印的提示」和「系统层面真正的错误」。
>
> 【出处】我们最典型的一类报错是配置写错了类型（该写小数的地方写了整数），
> 程序直接崩在启动阶段。**报错信息里其实写清楚了是哪个参数、错在哪**，
> 只是第一眼看不懂就容易被忽略。

C4.「想让它绕开障碍，它却倒车了」【核心 · 最推荐】

> 你写了一个「遇到障碍就换条路走」的程序。测试时发现，车不但没绕开，
> 反而**往后倒了一米多**，然后卡住不动。你觉得可能是什么原因？

> **想听到**：
>
> - 不是急着说「传感器坏了」，而是先问一句：**「程序是凭什么挑出这条路的？」**
> - 能想到「我给的规则里可能根本没禁止倒车」，或者「倒车在它的评分里恰好更划算」；
> - 意识到要去看**判断标准**，而不是盯着「我期望它选什么」。
>
> **减分**：直接跳到「雷达坏了」「代码写错了」，不做任何假设检验。
>
> 【出处】我们早期把「临时绕路」交给一个现成的规划器，它一路倒车 1 米然后原地卡死。
> 查下来根因是：它的评分里「尽量往前开」这一项权重只有 0.4，
> 而当时车头方向和目标差了 150°，**对它来说「倒车」比「原地掉头」更便宜**。
> 不是某个开关没开，是**它的评价标准和我们想的不一样**。

C5.「人眼明明有路，程序说没路」【核心 · 最推荐】

> 程序报「找不到能走的办法」。你把现场照片拿出来一看，
> **明明有一条很宽的空路能过去**。你会怎么想？

> **想听到**：
>
> - 关键分水岭：**「没路」是这个世界的事实，还是我所用的模型的结论？**
> - 会去问「程序是按什么标准判断『能走』的」——比如它可能要求「全程离原定路线不能超过 1 米」，而那条空路离得太远；也可能它能想到的方案只有固定的几种形状，根本生成不出那条路；
> - 愿意把「程序和现实不符」当成**自己的假设错了**，而不是「程序坏了」。
>
> **减分**：说「那就把限制放宽一点」——放宽之前得先知道是哪条限制在挡。
>
> 【出处】有一次我们的车在场地中间直接停住，日志写着「17 条候选方案全部被路径走廊拒绝」，
> 而现场看周围是空的。真正的原因是：这 17 条方案都必须往前推演 1.8 米，
> 可当时能参考的路径只剩 **0.116 米**——**不是没路，是它的模型表达不出那条路**。

C6.「改了参数，一点变化都没有」【进阶】

> 你改了一个参数，重新跑一遍，发现结果和改之前**一模一样**。你的第一反应是什么？

> **想听到**：先确认「**我跑的到底是不是我刚改的那份**」——
> 编译了吗？是不是有两套代码？实际运行的是哪个目录下的？改的是配置文件还是另一份副本？
> 确认完了，再去怀疑参数本身没用。
>
> 【出处】这是整个备赛期里我们踩得最多、也最浪费时间的一类问题：
> 改动只写在源码里，实际运行的是另一份已编译好的副本；或者改完忘了重新加载。
> **「我改了但没变化」，九成不是参数的锅。**

C7.「怎么证明真的变好了」【进阶】

> 你调了一个参数，**感觉**比之前好。老师问你：「你怎么知道真的变好了？」

> **想听到**：能说出「感觉不算数」——要有可比较的数据：同样条件下跑多次，
> 记录成功次数 / 用时 / 误差；而且要说得出**自己改的是哪一个参数**。
>
> 【出处】我们在交接文档里写过一句话：「没有这些记录的『体感变好 / 变差』，
> 很难区分到底是路径的问题、定位的问题，还是规划的问题。」
> 这题答得好，说明他有做实验的意识——这一条比会写代码更难得。

C8.「你盯着看的那个数字，可信吗」【进阶】

> 你靠屏幕上或日志里的一个数字，来判断系统正不正常。你怎么确定**这个数字本身可靠**？

> **想听到**：意识到「观测工具本身可能不代表真实状态」——
> 要去找一个**独立的**验证途径（直接看原始数据、换个方式测），而不是只信那一个指标。
> 加分：能想到「一个东西叫什么名字，和它实际是什么，可能是两回事」。
>
> 【出处】三个真实例子，都是同一类错：
> ① 界面上有个「车身方框」，它只在某个模块发指令时才刷新；它不动的时候我们以为定位挂了，
> 其实**那个框本来就不会动**；
> ② 日志里打印「候选点 = 30」，我们以为有 30 条候选方案，
> 实际是「参与计算的障碍点数 = 30」——**字段名把人骗了**；
> ③ 录包里一条数据都没有，我们以为是跑了旧程序，其实是那个模块
> **根本不往录包里写**，只打在终端上。

C9.「看不见了，就等于没有吗」【进阶】

> 你的传感器有个「盲区」——某个角度永远看不到。车往前走，一个障碍物先出现在视野里，
> 然后转进了盲区。这时候程序应该认为「那里还有障碍物」，还是「那里空了」？你会怎么设计？

> **想听到**：意识到「**看不见 ≠ 不存在**」，所以需要一个「记忆」；
> 以及记忆总得有个过期规则——多久之后才可以当作没有了？
>
> 【出处】我们用的单线激光雷达有盲区。如果直接把「这一帧没扫到」当成「没有障碍」，
> 车转个向就会撞上去。后来加了一层「记住最近一秒看到过的障碍」。

C10.「两个地方都在改同一个值」【进阶】

> 程序里有两个地方都会去「修正」同一个数值。跑一会儿你发现，
> 你在 A 处做的修正，过一段时间被 B 悄悄改回去了。你会怎么查？

> **想听到**：用「谁在什么时候改了这个值」的思路去找——
> 加打印看变化发生的时刻、把 B 临时关掉验证、找出所有会写这个值的地方。
>
> 【出处】我们做定位修正时，修正量只更新了一处状态，另一处用于匹配的参考点没同步更新，
> 结果下一帧数据一来就把修正覆盖掉了；还有一次是两个模块都在发布同一个坐标关系，
> 系统里出现了两个互相冲突的「父子关系」。

D. 动手小测（可选，5 分钟；如果现场有电脑）

> 这一节不是必做项。如果候选人是低年级、基础课还没学完，跳过 D 类，
> 把时间留给 C 类。

D1.（Python，现场写） 读一个文本文件，统计每个单词出现次数，打印出现最多的 3 个。

> 看的是：会不会查文档、变量命名清不清楚、边界情况（空文件、大小写、标点）想没想到。

D2.（逻辑） 有 8 个外表一样的小球，其中 1 个偏重，用天平最少几次找出来？

> 参考：2 次。看的是**会不会用「分组二分」的思路**，而不是硬数。

D3.（看代码） 给他 10 行我们项目里的代码（比如 `origincar_control.cpp` 里
巡线那一行 `v = (line_x - REAL_MIDX) * xunxian_k;`），问他这行在干什么。

> 看的是：能不能从变量名和算式猜出意图，以及会不会主动问「`REAL_MIDX` 是什么」。
> **会问问题的人是加分，不是减分。**

E. 协作与开放（3~5 分钟，挑 1 题）

E1. 你和队友意见不一致，但必须选一个方案，你会怎么做？

> 想听到：先把判断标准说清楚（哪个指标更重要），再用小实验去比，
> 或者请老师仲裁；而不是靠谁嗓门大。

E2. 你在项目里做过最「无聊但必须做」的事是什么？

> 想听到：整理文档、记录数据、反复测试这类事他做没做过。
> 我们队里一半的工作量都在这里。

E3. 如果这个学期你课业很重，你打算怎么安排备赛时间？

> 想听到：**具体的**时间安排，而不是「我会努力」。
> 减分：承诺「我每天都有空」——通常做不到，反而说明没规划过。

E4. 你和队友一起调同一个东西，他说他那边没问题，你这边有问题。你会怎么推进？

> 想听到：找**最小可复现的例子**，两个人对着同一个例子跑，
> 把不同的地方一条条列出来；而不是互相说「你那边肯定有问题」。
> （这题原来放在 C 类，但它考的是协作不是调试，所以挪到这里。）

E5. 你还有什么想问我们的？

> 想听到：问「平时怎么分工」「新人第一步做什么」「机器现在能跑到什么程度」
> 这类**真想知道**的问题。
> 减分：不问。也减分：只问「能加多少学分」「有没有奖金」。

评分表（打印一份，每人一张）

每项 1~5 分，1 = 明显不足，3 = 达到要求，5 = 超出预期。

维度	观察点	得分
基础扎实度	B 类答题情况：变量/文件/权限/浮点数这些概念清不清楚	/5
Debug 思维（权重最高）	C 类：会不会**分层排查**、有没有**可观测**意识、懂不懂**控制变量**	/5
动手经历	A1：有没有真的自己做过东西、自己踩过坑	/5
学习意愿与方法	答不上来时是慌还是想办法；会不会主动查、主动问	/5
沟通表达	能不能把一件事讲清楚，别人听懂	/5
稳定性与投入	E3：时间安排是否现实、动机是否具体	/5

加权建议：Debug 思维 ×2，其余 ×1。总分 35 分。
建议 24 分以上进入下一轮，但不要只看总分——
「基础 3 分 + Debug 5 分」通常比「基础 5 分 + Debug 2 分」更适合我们队。

面试官注意事项

1. **别问「你有没有做过机器人」。**大部分新生都没有，这个问题只是在筛家境和运气。
   改成问「你最近做出来过什么」，范围一下子打开了。
2. **别问八股。**比如「ROS 2 和 ROS 1 的区别」「卡尔曼滤波的五个公式」——
   这些进来学两周就会，答不出来不能说明任何事。
3. **答不上来时，给一点提示再看反应。**有人不会但一点就通，有人不会且完全不想动脑，
   这两种人的差别比「会不会」本身重要得多。
4. **追问比问题本身重要。**同一个问题，「你是背下来的还是自己想过的」一追就清楚。
5. **一定要问 E4。**愿不愿意主动问问题，是最能预测他后面会不会一直麻烦你的指标。
6. **不要当场承诺名额。**统一口径：「结果会在 X 天内通知」。

面试官速查卡（一页纸版本）

只留五句话，够用了：

1. **你最近自己做出来过什么？**（有没有真动过手）
2. **昨天能跑今天不动了，你怎么查？**（会不会分层排查）
3. **十次里错一次的问题，你怎么办？**（有没有复现和留证据的意识）
4. **人眼明明有路，程序说没路，你会怎么想？**（会不会怀疑自己的模型，而不是张口怪程序）
5. **答不上来时，你可以问我**（观察他会不会用人和资源）